Document Number: P0574R0
Date: 2017-02-04
Author: Anthony Williams
Just Software Solutions Ltd
Audience: SG1, LWG

P0574R0: Algorithm Complexity Constraints and Parallel Overloads

Introduction

The C++17 draft features overloads of a lot of the standard library algorithms which take an `ExecutionPolicy` argument, in order to allow parallel implementations. However, the requirements of many of the algorithms are written in such a way as to require a sequential implementation, or a sub-optimal parallel implementation, thus eliminating some or all of the benefit in the new overloads.

P0523r0: Wording for CH 10: Complexity of parallel algorithms attempts to address this problem by a blanket relaxation of the constraints for overloads with an `ExecutionPolicy`. I think this is an incorrect solution. It simultaneously grants too much leeway for some algorithms, while not fixing the problems with others. Instead, I think it is better to address each algorithm individually.

In this paper I have gathered those algorithms where the requirements are specified in such a way as to rule out an efficient parallel implementation, and proposed alternative specifications for those algorithms.

Algorithms and proposed wording

25.3.8 `adjacent_find` [alg.adjacent.find]

Change paragraph 2 as follows:

> Complexity: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, fFor a nonempty range, exactly `min((i - first) + 1, (last - first) - 1)` applications of the corresponding predicate, where `i` is `adjacent_find`'s return value. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, at most `(last-first)-1` applications of the corresponding predicate.

25.4.1 Copy [alg.copy]

Modify the requirements for `copy_if` in paragraph 12 as follows:

> Requires: The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, `std::iterator_traits<InputIterator>::value_type` shall be `MoveConstructible`.

25.6.8 Remove [alg.remove]

Modify the requirements for `remove_copy` and `remove_copy_if` in paragraph 7 as follows:

> Requires: The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap. The expression `*result = *first` shall be valid. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, the type of `*first` shall satisfy the `CopyConstructible` requirements.

25.6.9 Unique [alg.unique]

Modify the requirements for `unique_copy` and `unique_copy_if` in paragraph 5 as follows:

> Requires: The comparison function shall be an equivalence relation. The ranges `[first, last)` and `[result, result+(last-first))` shall not overlap. The expression `*result = *first` shall be valid. Let `T` be the value type of `InputIterator`. For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, the following shall hold:
>
> — If `InputIterator` meets the forward iterator requirements, then there are no additional requirements for `T`.

Otherwise, if `OutputIterator` meets the forward iterator requirements and its value type is the same as `T`, then `T` shall be `CopyAssignable` (Table 26). Otherwise, `T` shall be both `CopyConstructible` (Table 24) and `CopyAssignable`.

For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, `T` shall be both `CopyConstructible` (Table 24) and `CopyAssignable`.

25.6.14 Partitions [alg.partitions]

Modify the complexity for `partition` in paragraph 7 as follows:

Complexity: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, i~~I~~f `ForwardIterator` meets the requirements for a `BidirectionalIterator`, at most `(last - first) / 2` swaps are done; otherwise at most `last - first` swaps are done. Exactly `last - first` applications of the predicate are done. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, no worse than O(N log N) swaps, where N = `last - first`. Exactly `last - first` applications of the predicate are done.

Modify the complexity for `stable_partition` in paragraph 11 as follows:

Complexity: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, a~~A~~t most N log(N) swaps, where N = `last - first`, but only O(N) swaps if there is enough extra memory. Exactly `last - first` applications of the predicate. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, O(N log N) swaps, where N = `last - first`. Exactly `last - first` applications of the predicate are done.

Modify the requirements for `partition_copy` in paragraph 12 as follows:

Requires: `InputIterator`'s value type shall be `CopyAssignable`, and shall be writable (24.2.1) to the `out_true` and `out_false` `OutputIterator`s, and shall be convertible to `Predicate`'s argument type. The input range shall not overlap with either of the output ranges. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, `InputIterator`'s value type shall also be `CopyConstructible`.

25.7.2 Nth element [alg.nth.element]

Modify the complexity for `nth_element` in paragraph 3 as follows:

Complexity: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, l~~L~~inear on average. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, O(N) executions of the predicate, and no worse than O(N log N) swaps, where N = `last - first`.

25.7.4 Merge [alg.merge]

Modify the complexity for `merge` in paragraph 4 as follows:

Complexity: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, a~~A~~t most `(last1 - first1) + (last2 - first2) - 1` comparisons. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, O(`(last1 - first1) + (last2 - first2) - 1`) comparisons.

Modify the complexity for `inplace_merge` in paragraph 8 as follows:

Complexity: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, if~~When~~ enough additional memory is available, `(last - first) - 1` comparisons. ~~If no additional memory is available~~Otherwise, an algorithm with complexity at most O(`N` log(`N`)) may be used, where `N = last - first`.

26.8.3 Reduce [reduce]

Modify paragraph 5 as follows:

Requires: `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the range `[first, last)`. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, both `T` and `BinaryOperation` shall be `CopyConstructible`. All of `binary_op(init,*first)`, `binary_op(*first,init)`, `binary_op(init,init)`, and `binary_op(*first,*first)` shall be convertible to `T`.

26.8.4 Transform reduce [transform.reduce]

Modify paragraph 1 as follows:

Requires: Neither `unary_op` nor `binary_op` shall invalidate subranges, or modify elements in the range `[first, last)`. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, all of `T`, `UnaryOperation` and `BinaryOperation` shall be `CopyConstructible`. All of `binary_op(init,unary_op(*first),binary_op(unary_op(*first),init), binary_op(init,init)`, and `binary_op(unary_op(*first),unary_op(*first))` shall be convertible to `T`.

26.8.11 Adjacent difference [adjacent.difference]

Change paragraph 2 as follows:

Effects: For the overloads with no `ExecutionPolicy`, or an `ExecutionPolicy` of `std::execution::sequential_policy`, f~~F~~or a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, creates an object `val` whose type is `InputIterator`'s value type, initializes it with `*i`, computes `val - acc` or `binary_op(val, acc)`, assigns the result to `*(result + (i - first))`, and move assigns from `val` to `acc`. For the overloads with an `ExecutionPolicy` other than `std::execution::sequential_policy`, for a non-empty range, for every value `d` in `[1,last-first-1)` the function computes `*(first+d)-*(first+d-1)` or `binary_op(*(first+d),*(first+d-1))` and assigns the result to `*(result + d)`